

JSON

CHEAT SHEETS

A Complete Reference for Data Interchange & APIs

Syntax & Structure

Data Types

Objects & Arrays

JSON Schema

Parsing & Serialization

JSONPath & jq

REST APIs

JWT & GeoJSON

NDJSON & JSON5

8 REFERENCE SHEETS

CONTENTS

FUNDAMENTALS

- 01 Syntax & Structure
- 02 Values & Data Types
- 03 Objects & Arrays
- 04 JSON Schema

APPLIED

- 05 Parsing & Serialization
- 06 JSONPath & Querying
- 07 JSON in APIs & HTTP
- 08 Advanced & Related Formats

Covers [RFC 8259](#) • [JSON Schema Draft 2020-12](#) • [RFC 9535 JSONPath](#) • [jq](#) • [JWT](#) • [GeoJSON \(2025\)](#) [Begin](#) → [Sheet 01](#)

Sheet 1

Syntax & Structure

spec overview · key-value pairs · nesting ·
whitespace · encoding · escape sequences

Sheet 2

Values & Data Types

string · number · boolean · null · array · object · type
rules · edge cases

Sheet 3

Objects & Arrays

object syntax · array syntax · nested structures ·
mixed types · empty values · ordering

Sheet 4

JSON Schema

type validation · required · properties · \$ref · \$defs ·
allOf · anyOf · oneOf

Sheet 5

Parsing & Serialization

parse · stringify · pretty-print · error handling · JS ·
Python · PHP · Ruby · Go

Sheet 6

JSONPath & Querying

JSONPath syntax · dot notation · wildcards · filters ·
recursive descent · jq

Sheet 7

JSON in APIs & HTTP

REST conventions · Content-Type ·
request/response shapes · pagination · RFC 7807
errors

Sheet 8

Advanced & Related Formats

JSON5 · JSONC · NDJSON · BSON · JWT · GeoJSON ·
format selection guide

WHAT IS JSON?

JSON (JavaScript Object Notation) is a lightweight, text-based data-interchange format defined by **RFC 8259** and **ECMA-404**. Despite its name, JSON is completely language-independent and is the dominant format for REST APIs, config files, and data storage. A JSON document is always encoded in **UTF-8** (UTF-16 and UTF-32 are also permitted by the spec but rarely used in practice). Every valid JSON document is either a single *value* — an object, array, string, number, boolean, or null.

KEY RULES

- › Keys **must** be double-quoted strings — single quotes are invalid
- › Values must be one of the six types: object, array, string, number, boolean, or **null**
- › Trailing commas are **not** allowed after the last item in an object or array
- › Comments are **not** part of the spec — strip them before parsing
- › Whitespace (spaces, tabs, newlines) outside strings is insignificant
- › The root value may be any JSON type, not just an object

ANNOTATED JSON DOCUMENT

```
{
  "name": "Alice",
  "age": 30,
  "active": true,
  "score": 9.5,
  "nickname": null,
  "tags": ["admin", "editor"],
  "address": {
    "city": "Austin",
    "zip": "78701"
  }
}
```

LINE NOTES

- "name": "Alice" — string value; key and value both double-quoted
- "age": 30 — integer number; no quotes around the value
- "active": true — boolean literal; must be lowercase
- "score": 9.5 — floating-point number
- "nickname": null — null literal; must be lowercase
- "tags": [...] — array of strings as a value
- "address": {...} — nested object as a value

DOCUMENT STRUCTURE RULES

RULE	VALID	INVALID
Key quotes	"key": 1	key: 1
String quotes	"hello"	'hello'
Trailing comma	{"a":1}	{"a":1,}
Boolean case	true / false	True / FALSE
Null case	null	Null / NULL
Comments	— not supported —	// comment
Root type	any JSON value	bare identifier

ENCODING & ESCAPES

ESCAPE	CHARACTER
\"	Double quote
\\	Backslash
\/	Forward slash (optional)
\n	Newline
\r	Carriage return
\t	Tab
\uXXXX	Unicode code point (4 hex digits)

COMMON MISTAKES

- › Using single quotes for keys or strings — only double quotes are valid
- › Leaving a trailing comma after the last object property or array element
- › Writing **True**, **False**, or **None** (Python style) instead of lowercase JSON literals
- › Embedding comments with `//` or `/* */` — parsers will reject them

TIPS

- › Use a linter or validator (jsonlint.com, VS Code) to catch syntax errors before runtime
- › Prefer 2-space indentation for readability; minify for production transport
- › If you need comments in config files, consider **JSON5** or **JSONC** (see Sheet 08)

SUMMARY

- › JSON is defined by RFC 8259 — language-independent, UTF-8 text
- › Six value types: object · array · string · number · boolean · null
- › Keys must be double-quoted; no trailing commas; no comments
- › Booleans and null are lowercase literals: **true**, **false**, **null**

THE SIX JSON TYPES

JSON defines exactly **six value types**. There is no distinction between integers and floats — all numbers share a single type. There are no date, undefined, function, or binary types; these must be represented as strings or omitted. Understanding the limits of each type prevents silent data loss, especially with **large integers** and **special float values** like `Infinity` and `NaN`, which are not valid JSON.

TYPE QUICK RULES

- › `string` — always double-quoted; escape special chars with `\`
- › `number` — no `NaN`, no `Infinity`, no leading zeros (except `0.x`)
- › `boolean` — only `true` or `false` (lowercase)
- › `null` — represents absence of a value; distinct from `false` or `0`
- › `object` — unordered collection of key/value pairs wrapped in `{ }`
- › `array` — ordered list of values wrapped in `[ ]`; may mix types
- › JSON has no `undefined` — missing keys and `null` values are different things

ALL SIX TYPES IN ONE DOCUMENT

```
{
  "type_string": "Hello, World!",
  "type_escape": "Line 1\nLine 2\t(tab)",
  "type_unicode": "\u00e9\u00e8",

  "type_integer": 42,
  "type_float": 3.14159,
  "type_exp": 2.5e10,
  "type_neg": -7,

  "type_true": true,
  "type_false": false,

  "type_null": null,

  "type_array": [1, "two", true, null],

  "type_object": { "x": 1, "y": 2 }
}
```

NOTES

- `\n`, `\t` — escape sequences inside strings; raw newlines not allowed
- `\uXXXX` — any Unicode character by hex code point
- `2.5e10` — scientific notation is valid; equals 25000000000
- `-7` — negative numbers use a leading minus; no leading plus allowed
- Arrays may contain mixed types — no schema enforcement in bare JSON
- `null` ≠ `false` ≠ `0` ≠ `""` — each is a distinct value

NUMBER RULES & LIMITS

FORM	EXAMPLE	NOTES
Integer	42	No size limit in spec; parsers may differ
Negative	-3	Leading + is invalid
Float	3.14	Must have digit before decimal point
Exponent	1e8	e or E both valid
Leading zero	0.5	07 is invalid
NaN	— invalid —	Not in spec; use <code>null</code> instead
Infinity	— invalid —	Not in spec; use <code>null</code> or string

STRING RULES & EDGE CASES

CASE	VALID JSON
Empty string	<code>""</code>
Whitespace only	<code>" "</code>
Unicode	<code>"\u4e2d\u6587"</code>
Surrogate pair	<code>"\ud83d\ude00"</code>
Embedded quote	<code>"say \"hi\""</code>
Backslash	<code>"C:\\path"</code>
Raw newline in string	— invalid; use <code>\n</code> —

COMMON MISTAKES

- › Using `NaN` or `Infinity` in number fields — neither is valid JSON
- › Storing dates as numbers (Unix timestamps) without documenting the unit (ms vs s)
- › Assuming large integers are safe — parsers using IEEE 754 doubles lose precision above 2⁵³
- › Treating `null` and a missing key as equivalent — they have different semantics

TIPS

- › Represent dates as ISO 8601 strings: `"2024-06-15T14:30:00Z"`
- › For large integers (IDs, snowflakes), use strings to avoid precision loss across languages
- › Use `null` explicitly to distinguish "value is absent" from "value is empty string"

SUMMARY

- › Six types: `string` · `number` · `boolean` · `null` · `object` · `array`
- › Numbers: no `NaN`, no `Infinity`, no leading zeros (except `0.x`)
- › Strings must be double-quoted; use `\uXXXX` for Unicode escapes
- › Arrays are ordered and may mix types; objects are unordered key-value maps

OBJECTS & ARRAYS

A JSON **object** is an *unordered* collection of zero or more key/value pairs enclosed in curly braces `{ }`. Keys must be unique strings within one object (duplicate keys are technically undefined behaviour). A JSON **array** is an *ordered* sequence of zero or more values enclosed in square brackets `[ ]`. Arrays preserve insertion order and may contain any mix of the six JSON types, including other arrays or objects, enabling arbitrarily deep nesting.

KEY RULES

- Object key order is **not guaranteed** — never rely on it for logic
- Duplicate keys in one object: spec says undefined; most parsers keep the last value
- Array elements are accessed by zero-based index, not by key
- Empty object `{}` and empty array `[]` are both valid JSON values
- Arrays and objects can nest to any depth — but keep it manageable for humans
- A single comma separates each element; no comma after the last

OBJECTS, ARRAYS & NESTING

```
{
  "user": {
    "id": 1,
    "name": "Bob",
    "roles": ["admin", "editor"],
    "address": {
      "city": "Denver",
      "coords": [39.7392, -104.9903]
    }
  },

  "empty_obj": {},
  "empty_arr": [],

  "matrix": [
    [1, 2, 3],
    [4, 5, 6]
  ],

  "mixed": [42, "hello", true, null, {"k": "v"}]
}
```

NOTES

- `"roles": [...]` — array as an object value; common pattern for tags/permissions
- `"coords": [lat, lng]` — array of numbers; positional meaning is by convention
- `"empty_obj": {}` — valid; represents an object with no properties yet
- `"empty_arr": []` — valid; represents a list with zero items
- `"matrix"` — array of arrays; creates a 2D structure
- `"mixed"` — arrays can contain any type mix including nested objects

OBJECT PATTERNS

PATTERN	EXAMPLE	NOTES
Simple pair	<code>{"k": "v"}</code>	Minimal object
Multi-key	<code>{"a":1,"b":2}</code>	Comma-separated, no trailing comma
Nested object	<code>{"a":{"b":1}}</code>	Any depth
Array value	<code>{"tags":["x","y"]}</code>	Array as property value
Null value	<code>{"x": null}</code>	Key present, value absent
Empty	<code>{}</code>	Zero properties

ARRAY PATTERNS

PATTERN	EXAMPLE	NOTES
String list	<code>["a","b","c"]</code>	Ordered, indexed from 0
Number list	<code>[1,2,3]</code>	Homogeneous is conventional
Mixed types	<code>[1,"x",null]</code>	Spec allows; avoid if avoidable
Array of objects	<code>[{"id":1}, {"id":2}]</code>	Most common API response shape
Nested arrays	<code>[[1,2],[3,4]]</code>	Matrix / grid data
Empty	<code>[]</code>	Zero elements

COMMON MISTAKES

- Relying on object key order — it is not guaranteed by the spec or all parsers
- Using duplicate keys in the same object — behaviour is undefined and parser-dependent
- Trailing comma after last array element or object property
- Confusing positional array meaning — `[lat, lng]` vs `[lng, lat]` requires documentation

TIPS

- Prefer arrays of objects with named keys over positional arrays for clarity
- Use empty arrays `[]` rather than `null` for absent list values — easier to iterate
- Keep nesting to 3–4 levels maximum; deeper structures are a sign to redesign the schema

SUMMARY

- Objects: unordered key/value pairs in `{ }` — keys are unique strings
- Arrays: ordered values in `[ ]` — zero-indexed, mixed types allowed
- Both can be empty (`{ }`, `[]`) and nested to any depth
- Array of objects is the most common shape for list responses in REST APIs

WHAT IS JSON SCHEMA?

JSON Schema is a vocabulary (draft standardized at *json-schema.org*) for describing the structure and constraints of JSON documents. A schema is itself a JSON object containing keywords like `type`, `properties`, and `required`. Validators (ajv, jsonschema, etc.) use the schema to check whether a JSON instance conforms. Schemas are used for API documentation, form validation, code generation, and config file enforcement. The current stable release is **Draft 2020-12**.

CORE KEYWORDS

- › `$schema` — declares which draft is being used
- › `type` — constrains to one or more JSON types
- › `properties` — defines allowed keys and their sub-schemas for objects
- › `required` — array of keys that must be present
- › `$defs` — named reusable sub-schemas (replaces old `definitions`)
- › `$ref` — reference to another schema by JSON Pointer URI
- › `additionalProperties` — `false` to disallow unlisted keys

FULL SCHEMA EXAMPLE

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://example.com/user.schema.json",
  "title": "User",
  "type": "object",
  "required": ["id", "name", "email"],
  "properties": {
    "id": { "type": "integer", "minimum": 1 },
    "name": { "type": "string", "minLength": 1, "maxLength": 100 },
    "email": { "type": "string", "format": "email" },
    "age": { "type": "integer", "minimum": 0, "maximum": 150 },
    "role": { "$ref": "#/$defs/role" },
    "tags": { "type": "array", "items": { "type": "string" } }
  },
  "additionalProperties": false,
  "$defs": {
    "role": {
      "type": "string",
      "enum": ["admin", "editor", "viewer"]
    }
  }
}
```

NOTES

- `$schema` — tells validators which draft rules to apply
- `$id` — canonical URI for this schema; enables cross-schema `$ref`
- `"required": [...]` — keys listed here must exist in the instance
- `"minimum": 1` — number constraint; see table for all numeric keywords
- `"format": "email"` — semantic hint; validators may or may not enforce it
- `"$ref": "#/$defs/role"` — reuses the `role` definition inline
- `"additionalProperties": false` — rejects any key not listed in `properties`

VALIDATION KEYWORDS BY TYPE

TYPE	KEYWORD	MEANING
string	minLength / maxLength	Character count bounds
string	pattern	Regex the value must match
string	format	Semantic format hint (email, uri, date...)
string	enum	Value must be one of the listed options
number	minimum / maximum	Inclusive bounds
number	exclusiveMinimum	Value must be strictly greater
number	multipleOf	Value must be a multiple of N
array	items	Schema applied to every element
array	minItems / maxItems	Array length bounds
array	uniqueItems	true to require distinct elements
object	required	Array of mandatory key names
object	additionalProperties	false = no extra keys allowed

COMMON MISTAKES

- › Confusing `anyOf` and `oneOf` — use `oneOf` only when exactly one branch should match
- › Omitting `additionalProperties: false` then being surprised extra keys are accepted
- › Using old `definitions` key — it was replaced by `$defs` in Draft 2019-09
- › Assuming `format` is enforced — most validators treat it as advisory by default

TIPS

- › Use `$defs` to DRY up repeated sub-schemas and reference with `$ref`
- › Set `"additionalProperties": false` on API payloads to catch typos in field names
- › Test schemas with `jsonschema.dev` or `ajv-cli` before deploying

SUMMARY

- › JSON Schema validates JSON structure using keyword-based rules
- › `type` · `properties` · `required` are the three most-used keywords
- › `$defs` + `$ref` enable reusable, composable schemas
- › `allOf` / `anyOf` / `oneOf` combine schemas with AND / OR / XOR logic

PARSE VS SERIALIZE

Parsing (deserializing) converts a JSON string into a native data structure — a dictionary, map, object, or hash depending on the language. **Serialization** (stringifying / marshalling) is the reverse: converting native data into a JSON string. Every major language ships a JSON library in its standard library. The table below shows the canonical function names; all do the same thing under different names.

UNIVERSAL CONCEPTS

- › **Parse** — JSON string → native object/dict/map
- › **Stringify** — native value → JSON string
- › **Pretty-print** — serialize with indentation for readability
- › Always wrap parse calls in error handling — malformed JSON throws
- › Circular references in native objects cannot be serialized to JSON
- › **undefined** (JS), functions, and binary data are stripped on serialize

PARSE & SERIALIZE — FOUR LANGUAGES

```
// — JavaScript —
const obj = JSON.parse('{"name":"Alice","age":30}');
const str = JSON.stringify(obj);           // compact
const pretty = JSON.stringify(obj, null, 2); // indented

// — Python —
# import json
obj = json.loads('{"name":"Alice","age":30}')
str = json.dumps(obj)           # compact
pretty = json.dumps(obj, indent=2) # indented

// — PHP —
$obj = json_decode('{"name":"Alice","age":30}', true);
$str = json_encode($obj);           // compact
$pretty = json_encode($obj, JSON_PRETTY_PRINT); // indented

// — Ruby —
# require 'json'
obj = JSON.parse('{"name":"Alice","age":30}')
str = obj.to_json           # compact
pretty = JSON.pretty_generate(obj) # indented
```

NOTES

- JS `JSON.parse` → throws `SyntaxError` on bad input; wrap in try/catch
- JS `JSON.stringify(obj, null, 2)` — second arg is a replacer, third is indent spaces
- Python `json.loads` → raises `json.JSONDecodeError` on bad input
- Python `json.dumps(obj, indent=2)` also accepts `sort_keys=True` for stable output
- PHP `json_decode(..., true)` — second arg `true` returns assoc array, not stdClass
- Ruby `JSON.parse` → raises `JSON::ParserError`; rescue it explicitly

PARSE & STRINGIFY BY LANGUAGE

LANGUAGE	PARSE (STRING → NATIVE)	STRINGIFY (NATIVE → STRING)
JavaScript	<code>JSON.parse(str)</code>	<code>JSON.stringify(val)</code>
Python	<code>json.loads(str)</code>	<code>json.dumps(val)</code>
PHP	<code>json_decode(str, true)</code>	<code>json_encode(val)</code>
Ruby	<code>JSON.parse(str)</code>	<code>val.to_json</code>
Java	<code>gson.fromJson(str, T)</code>	<code>gson.toJson(val)</code>
Go	<code>json.Unmarshal([]byte, &v)</code>	<code>json.Marshal(v)</code>
Rust	<code>serde_json::from_str(str)</code>	<code>serde_json::to_string(v al)</code>
C#	<code>JsonSerializer.Deserialize<T>(str)</code>	<code>JsonSerializer.Serialize(val)</code>

ERROR TYPES BY LANGUAGE

LANGUAGE	PARSE ERROR TYPE
JavaScript	<code>SyntaxError</code>
Python	<code>json.JSONDecodeError</code>
PHP	returns null; check <code>json_last_error()</code>
Ruby	<code>JSON::ParserError</code>
Java (Gson)	<code>JsonSyntaxException</code>
Go	<code>*json.SyntaxError</code>
Rust	<code>serde_json::Error</code>
C#	<code>JsonException</code>

COMMON MISTAKES

- › Not catching parse errors — malformed input from an API will crash unguarded code
- › PHP: forgetting the `true` second arg to `json_decode` and getting a stdClass instead of array
- › Serializing objects with circular references — all languages will error or loop infinitely
- › Using pretty-print in production API responses unnecessarily — adds bytes and latency

TIPS

- › Always validate + parse in one step — never manipulate raw JSON strings with regex
- › Use `sort_keys=True` (Python) or replacer sort in JS for deterministic, diffable output
- › For streaming large files, prefer streaming parsers (Python `ijson`, Node.js `stream-json`)

SUMMARY

- › Parse = JSON string → native; Stringify = native → JSON string
- › Every major language has a built-in JSON library — no third-party package needed for basics
- › Always handle parse errors — invalid JSON throws or returns null depending on language
- › Pretty-print with indent option for debugging; compact for production transport

JSONPATH

JSONPath (RFC 9535, 2024) is a query language for extracting values from JSON documents, analogous to XPath for XML. An expression always starts with **\$** (the root node) and chains selectors with either **dot notation** (`$.store.book`) or **bracket notation** (`$["store"]["book"]`). Both forms are equivalent. JSONPath is supported natively in PostgreSQL, MySQL, AWS Step Functions, many API gateways, and libraries for every major language.

SYNTAX QUICK REFERENCE

- › `$` — root of the document
- › `.key` — child property by name
- › `[n]` — array element at index *n* (zero-based)
- › `[*]` or `.*` — all children (wildcard)
- › `..` — recursive descent (all descendants)
- › `[?(@.price < 10)]` — filter expression
- › `[1:3]` — array slice (elements 1 and 2)

JQ — COMMAND-LINE JSON PROCESSOR

```
# — Basic access —
echo '{"name":"Alice","age":30}' | jq '.name'      # "Alice"
echo '{"name":"Alice","age":30}' | jq '.age'      # 30

# — Arrays —
echo '[1,2,3]' | jq '.[0]'                       # 1
echo '[1,2,3]' | jq '[]'                         # 1 2 3 (each on own line)
echo '[1,2,3]' | jq '[1:3]'                      # [2,3]

# — Filters & pipes —
cat data.json | jq '.users[] | .name'            # each user's name
cat data.json | jq '.users[] | select(.age > 25)' # filter by condition
cat data.json | jq '[.users[] | .name]'          # collect into array

# — Transform —
cat data.json | jq '{id: .id, label: .name}'      # reshape object
cat data.json | jq '.users | length'            # count items
cat data.json | jq '.users | sort_by(.age)'      # sort array
cat data.json | jq '.users | map(.age) | add'    # sum all ages

# — Output options —
jq -r '.name'      # raw output (no quotes around strings)
jq -c '.'          # compact (minified) output
jq '.' file.json   # pretty-print a file
```

NOTES

- `.` — identity filter; outputs the input unchanged
- `.key` — accesses object property; `.a.b` chains
- `[]` — iterates every element of an array or object
- `|` — pipe: output of left becomes input of right
- `select(cond)` — keeps input only if condition is true
- `[expr]` — collect filter output into an array
- `-r` flag — strips surrounding quotes from string output

JSONPATH EXPRESSION REFERENCE

EXPRESSION	SELECTS
<code>\$</code>	Root node
<code>\$.store</code>	Child store of root
<code>\$.store.book[0]</code>	First book
<code>\$.store.book[*]</code>	All books
<code>..author</code>	All author values anywhere
<code>\$.store.book[-1]</code>	Last book (RFC 9535)
<code>\$.store.book[0,1]</code>	First and second book
<code>\$.store.book[?(@.price<10)]</code>	Books cheaper than 10
<code>\$.store.book[?(@.isbn)]</code>	Books that have an isbn key
<code>..price</code>	All price values in document

JQ BUILT-IN FUNCTIONS

FUNCTION	PURPOSE
<code>length</code>	Count elements or string chars
<code>keys</code>	Array of object keys, sorted
<code>values</code>	Array of object values
<code>has("k")</code>	True if key exists
<code>map(f)</code>	Apply filter to each element
<code>select(f)</code>	Keep input if filter is true
<code>sort_by(.k)</code>	Sort array by key
<code>group_by(.k)</code>	Group array by key value
<code>unique_by(.k)</code>	Deduplicate by key
<code>add</code>	Sum / concatenate array elements
<code>to_entries</code>	Object → [{key,value}] array
<code>from_entries</code>	[{key,value}] → object

COMMON MISTAKES

- › Forgetting `$` at the start of a JSONPath expression — it's mandatory
- › jq: omitting `-r` when piping string output to shell — the extra quotes break commands
- › Mixing JSONPath implementations — filter syntax varies between tools (Goessner vs RFC 9535)
- › Using `..` (recursive descent) on large documents — it can be slow on deep structures

TIPS

- › Test JSONPath expressions at [jsonpath.com](#) before embedding in code
- › Use `jq -c` to produce compact JSON for piping into other tools or curl
- › In jq, wrap multi-step transforms in a function with `def name: expr;` to keep scripts readable

SUMMARY

- › JSONPath: `$` is root · `.` is child · `[*]` is wildcard · `..` is recursive
- › Filter expressions `[?(@.field op value)]` select matching array elements
- › jq is the standard CLI tool — pipe-based, filter-oriented, and very fast
- › jq's `map`, `select`, `sort_by`, and `group_by` cover most data-shaping tasks

JSON AS THE API WIRE FORMAT

JSON is the *de-facto* wire format for REST and most HTTP APIs. The client signals it is sending JSON with `Content-Type: application/json` and requests JSON back with `Accept: application/json`. Well-designed APIs return consistent envelope shapes — a data wrapper for success responses and a structured error object for failures — so clients can handle both cases with the same parsing logic.

HTTP HEADER RULES

- › `Content-Type: application/json` — set on any request body you send
- › `Accept: application/json` — tells the server what format you want back
- › `Content-Type: application/json; charset=utf-8` — explicit encoding (usually redundant)
- › GET requests never have a body — pass data in query params, not JSON
- › 201 Created should return the newly created resource in the body
- › 204 No Content returns an empty body — don't try to parse it

COMMON API JSON SHAPES

```
// — Success: single resource —————
{
  "data": { "id": 1, "name": "Alice", "email": "alice@example.com" }
}

// — Success: collection with pagination —————
{
  "data": [
    { "id": 1, "name": "Alice" },
    { "id": 2, "name": "Bob" }
  ],
  "meta": {
    "total": 200,
    "page": 1,
    "per_page": 20,
    "next": "/users?page=2"
  }
}

// — Error response (RFC 7807 Problem Details) —————
{
  "type": "https://example.com/errors/validation",
  "title": "Validation Failed",
  "status": 422,
  "detail": "The 'email' field is not a valid address.",
  "errors": [
    { "field": "email", "message": "Invalid format" }
  ]
}
```

NOTES

- `"data"` envelope — wrapping the payload lets you add metadata later without breaking clients
- `"meta"` block — pagination, counts, and cursor values live here, not inside `data`
- `"next"` cursor — cursor-based pagination is safer than page numbers for live data
- RFC 7807 error shape — `type`, `title`, `status`, `detail` are the standard fields
- `"errors"` array — per-field validation errors alongside the top-level error message
- Always return JSON error bodies even on 4xx/5xx — never let plain-text stack traces leak

HTTP METHODS & JSON BODY RULES

METHOD	HAS BODY?	TYPICAL JSON USE
GET	No	Returns resource or collection in response
POST	Yes	Create; send new resource data in body
PUT	Yes	Full replace; send complete resource
PATCH	Yes	Partial update; send only changed fields
DELETE	Rarely	Usually no body; 204 or deleted resource back

COMMON STATUS CODES & JSON BODY

CODE	MEANING	BODY
200	OK	Requested data
201	Created	New resource
204	No Content	Empty — do not parse
400	Bad Request	Error + validation details
401	Unauthorized	Auth error message
403	Forbidden	Permission error
404	Not Found	Error message
422	Unprocessable	Validation errors array
429	Rate Limited	Retry-After info
500	Server Error	Generic error — no internals

COMMON MISTAKES

- › Returning a 200 status with an error message in the body — use the correct 4xx/5xx code
- › Forgetting `Content-Type: application/json` on POST/PUT — server may reject or misparse the body
- › Trying to parse a 204 No Content response body — it is empty by definition
- › Exposing stack traces or internal error messages in 500 responses

TIPS

- › Always check the HTTP status code before parsing the body — don't assume success
- › Adopt RFC 7807 Problem Details for error responses — it's becoming the industry standard
- › Use cursor-based pagination (`"next_cursor"`) instead of page numbers for stable, live data sets

SUMMARY

- › Set `Content-Type: application/json` on every request with a body
- › Wrap responses in a `data` envelope to allow adding `meta` later
- › Use RFC 7807 for consistent, machine-readable error responses
- › Check HTTP status code first — never trust a 200 blindly

THE JSON ECOSYSTEM

JSON's strict syntax makes it reliable but sometimes inconvenient — no comments, no trailing commas, no binary data. The ecosystem has produced several **supersets and relatives** that solve specific pain points. **JSON5** adds comments and unquoted keys for config files. **NDJSON** enables streaming of record sequences. **BSON** adds binary types for database storage. **JWT** uses Base64-encoded JSON for authentication tokens. Understanding which format to reach for — and why — is as important as knowing JSON itself.

FORMAT SELECTION GUIDE

- Config files with comments → **JSON5** or **JSONC**
- Streaming / log lines → **NDJSON** / **JSON Lines**
- Database storage with binary data → **BSON** (MongoDB)
- Auth tokens in HTTP headers → **JWT**
- Geographic features → **GeoJSON** (RFC 7946)
- REST API payloads → plain **JSON** (RFC 8259)
- High-performance binary APIs → **MessagePack** or **CBOR**

FORMAT EXAMPLES

```
// — JSON5 (config file) —————
{
  // Comments are allowed
  name: 'Alice',      // unquoted keys
  score: +9.5,        // leading + allowed
  hex: 0xFF,          // hex literals
  multiline: "line 1\
line 2",             // backslash line continuation
  trailing: "ok",     // trailing comma OK
}

// — NDJSON / JSON Lines (one object per line) —————
{"id":1,"event":"login","ts":"2024-01-01T00:00:00Z"}
{"id":2,"event":"view","ts":"2024-01-01T00:01:00Z"}
{"id":3,"event":"logout","ts":"2024-01-01T00:05:00Z"}

// — JWT structure (header.payload.signature) —————
// Header (Base64url-decoded):
{"alg":"HS256","typ":"JWT"}
// Payload (Base64url-decoded):
{"sub":"1234567890","name":"Alice","iat":1516239022}
// Full token:
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiIxMjM0In0.SflKxwR...

// — GeoJSON (RFC 7946) —————
{
  "type": "Feature",
  "geometry": {
    "type": "Point",
    "coordinates": [-104.9903, 39.7392]
  },
  "properties": { "name": "Denver" }
}
```

- ### NOTES
- JSON5 is a superset — all valid JSON is valid JSON5, but not vice versa
 - NDJSON: newline-delimited — stream parsers read one line at a time, no full-document parse needed
 - JWT payload is Base64url encoded, *not* encrypted — never put secrets in it without encryption
 - GeoJSON coordinates are always [\[longitude, latitude\]](#) — longitude first (common gotcha)
 - BSON adds types: Date, ObjectId, Binary, Decimal128 — only used with BSON-aware drivers (MongoDB)
 - JSONC (JSON with Comments) is used by VS Code [settings.json](#) and TypeScript [tsconfig.json](#)

JSON-RELATED FORMATS COMPARISON			
FORMAT	SUPERSET?	COMMENTS	PRIMARY USE
JSON	—	✗	APIs, data interchange
JSON5	✓	✓	Human-written config files
JSONC	✓	✓	IDE configs (VS Code, TS)
NDJSON	✓	✗	Log streaming, bulk import
JSON Lines	✓	✗	Same as NDJSON (alias)
BSON	No (binary)	✗	MongoDB wire protocol
JWT	No (encoded)	✗	Auth tokens in HTTP headers
GeoJSON	✓	✗	Geographic data (RFC 7946)

GEOJSON GEOMETRY TYPES	
TYPE	COORDINATES SHAPE
Point	[lng, lat]
MultiPoint	[[lng, lat], ...]
LineString	[[lng, lat], [lng, lat], ...]
Polygon	[[[lng, lat], ...]] — first/last same
MultiPolygon	Array of Polygon coord arrays
Feature	geometry + arbitrary properties
FeatureCollection	Array of Feature objects

COMMON MISTAKES

- Treating JWT as encrypted — the payload is only Base64-encoded and is publicly readable
- GeoJSON: using [\[lat, lng\]](#) order — the spec requires [\[longitude, latitude\]](#)
- Feeding NDJSON to a standard JSON parser — it will fail; parse line-by-line instead
- Using JSON5 in a production API — most HTTP clients and servers expect strict JSON

TIPS

- Use NDJSON for log ingestion pipelines — append a line per event, no file rewriting needed
- Inspect JWT payloads at [jwt.io](#) — paste the token to decode and verify
- For config files that developers edit directly, JSON5 or JSONC is friendlier than strict JSON

SUMMARY

- JSON5 / JSONC add comments & trailing commas — good for config, not APIs
- NDJSON: one JSON object per line — built for streaming and log pipelines
- JWT: Base64url-encoded JSON header + payload — not encrypted by default
- GeoJSON: standard geographic format — coordinates are always [\[lng, lat\]](#)